PR 2

Software Architecture

Alejandro Pérez Bueno

May 15, 2026

Table of Contents

Exercise 1 — Implementation

Overview

Three microservices have been fully implemented as part of the Photo&Film4You platform:

- **Course** (port 18084) — manages courses, enrollments and academic grading
- **Microcredential** (port 18085) — manages credential requests, approval and rejection
- **Notification** (port 18083) — event-driven; listens to Kafka and simulates email delivery

All services follow a hexagonal architecture (domain / application / infrastructure layers), use Spring Boot 2.6.2, PostgreSQL with Hibernate `create-drop`, and Apache Kafka for asynchronous messaging.

Course Microservice

Ten operations are exposed via `CourseRESTController`:

Method	Path	Operation
`GET`	`/courses`	findCourses (optional `?title=` / `?instructor=`)
`GET`	`/courses/{id}`	getCourseById
`POST`	`/courses`	createCourse
`PUT`	`/courses/{id}`	modifyCourseDetails
`PATCH`	`/courses/{id}/enrollments/open`	openEnrollment
`PATCH`	`/courses/{id}/enrollments/close`	closeEnrollment
`POST`	`/courses/{id}/enrollments`	enrollInCourse
`GET`	`/courses/{id}/students`	getEnrolledStudents
`PATCH`	`/courses/{id}/grade-reports/close`	closeGradeReports
`PATCH`	`/courses/{id}/close`	closeCourse

`closeGradeReports` marks enrollments with `qualification > 0` as `GRADED`, then calls `POST /microcredentials/{courseId}/create` on the Microcredential service to batch-create credentials for every graded enrollment.

An additional helper endpoint `GET /courses/{id}/enrollments` is exposed to support enrollment inspection and is required internally by the Microcredential service to resolve student email addresses.

Key corrections to the skeleton:

- `CourseStatus`: removed the erroneous `PENDING_CLOSUE` value. Final enum: `DRAFT`, `ENROLLMENT_OPEN`, `ACTIVE`, `CLOSED`.
- `Course`: default status changed from `ACTIVE` to `DRAFT`.

- `CourseRepositoryImpl`: added `.withIgnorePaths("status")` to the `ExampleMatcher` so `GET /courses` without filters returns all courses regardless of status.
- `EnrollmentEntity.toDomain()`: added `courseId` mapping, required by the Microcredential service for email lookup.
- `ApplicationConfiguration`: new file providing the `RestTemplate` bean used by `CourseServiceImpl`.

Microcredential Microservice

Six operations are exposed via `MicrocredentialRESTController`:

Method	Path	Operation
`POST`	`/microcredentials`	requestMicrocredential
`GET`	`/microcredentials/pending`	getPendingMicrocredentialRequests
`GET`	`/microcredentials/{id}`	getMicrocredentialById
`POST`	`/microcredentials/{courseId}/create`	createMicrocredentials
`PATCH`	`/microcredentials/{id}/approve`	approvePendingMicrocredential
`PATCH`	`/microcredentials/{id}/reject`	rejectPendingMicrocredential

Student email is **not** stored on the `Microcredential` entity. Instead, `courseId` is stored and the email is resolved at approve/reject time by calling `GET /courses/{courseId}/enrollments` on the Course service (Option B).

Every `REQUESTED` state transition publishes a Kafka event on `microcredential.microcredential_pending`. Approval publishes on `microcredential.microcredential_approved`; rejection on `microcredential.microcredential_rejec`

Key changes to the skeleton:

- `MicrocredentialRequest`: fields changed from `{submitDate, assignmentDate}` to `{enrollmentId, courseId, content}`.
- `Microcredential` / `MicrocredentialEntity`: added `courseId` field; removed `userEmail`.
- `MicrocredentialMessage`: added `@NoArgsConstructor` for Kafka deserialization.
- `GetEnrollmentResponse`: new DTO for deserializing Course service enrollment responses.

Notification Microservice

Four Kafka listeners are implemented in `KafkaClassListener` / `NotificationServiceImpl`. Email sending is simulated with `INFO`-level log lines.

Topic	Listener	Action
`product.unit_available`	`productAvailable`	Notifies users with an active alert for the product

Topic	Listener	Action
`microcredential.microcredential_pending`	`pendingCredential`	Notifies all `ADMIN` users of the pending microcredential
`microcredential.microcredential_approved`	`approvedCredential`	Notifies the student that their credential was granted
`microcredential.microcredential_rejected`	`rejectedCredential`	Notifies the student that their credential was rejected

Key changes to the skeleton:

- `NotificationServiceImpl`: implemented all four `notify*` methods; injected `RestTemplate` via constructor.
- `GetUserResponse`: added `type` field to allow filtering `ADMIN` users.
- `ApplicationConfiguration`: new file providing the `RestTemplate` bean.
- `application.properties`: removed the unused `microcredentialService.getMicrocredentialDetails.url` property.

Exercise 2 — Operation Evidence

All operations were executed against locally running services with a clean seed database. Infrastructure was started with `docker compose up -d` from the `spring-2026/` directory. No changes were made to the provided `docker-compose.yml`.

Services were started with:

```
java -jar course-0.0.1-SNAPSHOT.jar
java -jar microcredential-0.0.1-SNAPSHOT.jar
java -jar notification-0.0.1-SNAPSHOT.jar
```

The Postman collection `PRAC2_SA_Photo4You.postman_collection.json` was imported from the `/v3/api-docs` endpoint of each service and used to execute all requests below.

Course Service

C1 — findCourses (no filter)

Request

`GET http://localhost:18084/courses`

Response — HTTP 200


```
[
  {
    "id": 1, "title": "Digital Editing",
    "instructor": "instructor1@uoc.edu", "status": "CLOSED",
    "mode": "Face-to-face course", "price": 1000,
    "duration": 120, "language": "English", "location": "London"
  },
  {
    "id": 2, "title": "Night Photography",
    "instructor": "instructor2@uoc.edu", "status": "ENROLLMENT_OPEN",
    "mode": "Online course", "price": 800,
    "duration": 200, "language": "English", "location": "on the net"
  },
  {
    "id": 3, "title": "Sports Photography",
    "instructor": "instructor3@uoc.edu", "status": "ACTIVE",
    "mode": "Online course", "price": 900,
    "duration": 100, "language": "English", "location": "on the net"
  }
```

```
]
```

C2 — findCourses (filter by title)

Request

```
GET http://localhost:18084/courses?title=Photography
```

Response — HTTP 200

```
[
  {
    "id": 2, "title": "Night Photography",
    "instructor": "instructor2@uoc.edu", "status": "ENROLLMENT_OPEN"
  },
  {
    "id": 3, "title": "Sports Photography",
    "instructor": "instructor3@uoc.edu", "status": "ACTIVE"
  }
]
```

C3 — findCourses (filter by instructor)

Request

```
GET http://localhost:18084/courses?instructor=instructor2@uoc.edu
```

Response — HTTP 200

```
[
  {
    "id": 2, "title": "Night Photography",
    "instructor": "instructor2@uoc.edu", "status": "ENROLLMENT_OPEN"
  }
]
```

C4 — getCourseById

Request

```
GET http://localhost:18084/courses/2
```

Response — HTTP 200


```
{
  "id": 2,
  "instructor": "instructor2@uoc.edu",
  "title": "Night Photography",
  "description": "course about Night Photography",
  "enrollmentStartDate": "2025-12-31T23:00:00.000+00:00",
  "enrollmentEndDate": "2026-05-29T22:00:00.000+00:00",
  "mode": "Online course",
  "price": 800,
  "objectives": "NP Qualification",
  "methology": "methology 2",
  "duration": 200,
  "language": "English",
  "location": "on the net",
  "status": "ENROLLMENT_OPEN"
}
```

C5 — createCourse

Request

```
POST http://localhost:18084/courses
Content-Type: application/json
```


```
{
  "instructor": "instructor1@uoc.edu",
  "title": "Advanced Colour Grading",
  "description": "In-depth course on colour grading for photo and film production.",
  "enrollmentStartDate": "2026-09-01T00:00:00.000Z",
  "enrollmentEndDate": "2026-09-30T00:00:00.000Z",
  "mode": "Online",
  "price": 350,
  "objectives": "Master colour grading workflows in DaVinci Resolve and Lightroom.",
  "methology": "Video lectures, hands-on projects and weekly live Q&A sessions.",
  "duration": 80,
  "language": "English",
  "location": "Online"
}
```


Response — HTTP 201

4

C6 — modifyCourseDetails

Request

```
PUT http://localhost:18084/courses/2
Content-Type: application/json
```


```
{
  "instructor": "instructor2@uoc.edu",
  "title": "Night Photography - Advanced Edition",
  "description": "Extended version with long-exposure and astrophotography modules.",
  "enrollmentStartDate": "2026-01-01T00:00:00.000Z",
  "enrollmentEndDate": "2026-06-30T00:00:00.000Z",
  "mode": "Hybrid",
  "price": 250,
  "objectives": "Master night-sky photography and time-lapse composition.",
  "methology": "Weekly workshops with night shooting field sessions.",
  "duration": 60,
  "language": "English",
  "location": "Barcelona"
}
```


Response — HTTP 200

2

C7 — openEnrollment

Request

```
PATCH http://localhost:18084/courses/3/enrollment/open
      ?startDate=2026-05-15&endDate=2026-06-15
```

Response — HTTP 200

```
(empty body)
```

C8 — enrollInCourse

Request

`POST http://localhost:18084/courses/3/enrollments?email=user1@uoc.edu`

Response — HTTP 201

6

C9 — getEnrollmentsByCourse

Request

`GET http://localhost:18084/courses/3/enrollments`

Response — HTTP 200


```
[
  {
    "id": 5, "student": "user3@uoc.edu",
    "enrollmentDate": "2025-12-30T23:00:00.000+00:00",
    "qualification": 0, "status": "ACTIVE", "courseId": 3
  },
  {
    "id": 6, "student": "user1@uoc.edu",
    "enrollmentDate": "2026-05-15T10:49:13.963+00:00",
    "qualification": 0, "status": "ACTIVE", "courseId": 3
  }
]
```

C10 — getEnrolledStudents

Request

`GET http://localhost:18084/courses/3/students`

Response — HTTP 200 *(ACTIVE and GRADED enrollments only)*


```
[
  {
    "id": 5, "student": "user3@uoc.edu",
    "qualification": 0, "status": "ACTIVE", "courseId": 3
  },
  {
    "id": 6, "student": "user1@uoc.edu",
    "qualification": 0, "status": "ACTIVE", "courseId": 3
```

```
  }
]
```

C11 — closeEnrollment

Request

`PATCH http://localhost:18084/courses/3/enrollment/close`

Response — HTTP 200

`(empty body)`

Course id=3 status transitions from `ENROLLMENT_OPEN` → `ACTIVE`.

C12 — closeGradeReports

Request

`PATCH http://localhost:18084/courses/2/grade-reports/close`

Response — HTTP 200

`(empty body)`

Enrollments with `qualification > 0` in course 2 are promoted to `GRADED`. The service then calls `POST /microcredentials/2/create` on the Microcredential service, which auto-creates credentials for each graded enrollment and publishes Kafka pending-notification events.

C13 — closeCourse

Request

`PATCH http://localhost:18084/courses/1/close`

Response — HTTP 200

`(empty body)`

Course id=1 (Digital Editing) status is set to `CLOSED`.

Microcredential Service

M1 — requestMicrocredential (enrollment 3)

Request

```
POST http://localhost:18085/microcredentials
Content-Type: application/json
```


```
{
  "enrollmentId": 3,
  "courseId": 2,
  "content": "Microcredential awarded for successful completion of Night Photography (course 2), enrollmen
}
```


Response — HTTP 201

```
1
```

A Kafka event is published on `microcredential.microcredential_pending`. The Notification service logs an INFO message to all admins.

M2 — requestMicrocredential (enrollment 4)

Request

```
POST http://localhost:18085/microcredentials
Content-Type: application/json
```


```
{
  "enrollmentId": 4,
  "courseId": 2,
  "content": "Microcredential awarded for successful completion of Night Photography (course 2), enrollmen
}
```


Response — HTTP 201

```
2
```

M3 — getPendingMicrocredentialRequests

Request

```
GET http://localhost:18085/microcredentials/pending
```

Response — HTTP 200

```
[
  {
    "id": 1,
    "submitDate": "2026-05-15T10:49:14.341+00:00",
    "assignmentDate": "2026-05-15T10:49:14.341+00:00",
    "status": "REQUESTED",
    "content": "Microcredential awarded for successful completion of Night Photography (course 2), enrollm
    "enrollment": 3,
    "courseId": 2
  },
  {
    "id": 2,
    "submitDate": "2026-05-15T10:49:14.466+00:00",
    "assignmentDate": "2026-05-15T10:49:14.466+00:00",
    "status": "REQUESTED",
    "content": "Microcredential awarded for successful completion of Night Photography (course 2), enrollm
    "enrollment": 4,
    "courseId": 2
  }
]
```

M4 — getMicrocredentialById

Request

`GET http://localhost:18085/microcredentials/1`

Response — HTTP 200


```
{
  "id": 1,
  "submitDate": "2026-05-15T10:49:14.341+00:00",
  "assignmentDate": "2026-05-15T10:49:14.341+00:00",
  "status": "REQUESTED",
  "content": "Microcredential awarded for successful completion of Night Photography (course 2), enrollmen
  "enrollment": 3,
  "courseId": 2
}
```

M5 — requestCourseMicrocredentials

Request

```
POST http://localhost:18085/microcredentials/2/create
```

Response — HTTP 201

```
[]
```

The service fetches all enrollments for course 2 from the Course service and filters to `GRADED` status. The seed enrollments for course 2 have `qualification = 0` (no grading has occurred), so no microcredentials are created. The empty list is the correct and expected response for this database state. When `closeGradeReports` is called on a course that has enrollments with `qualification > 0`, those enrollments are set to `GRADED` and this endpoint produces a non-empty list.

M6 — approvePendingMicrocredential

Request

```
PATCH http://localhost:18085/microcredentials/1/approve
```

Response — HTTP 200

```
1
```

Microcredential id=1 transitions from `REQUESTED` → `GRANTED`. The service resolves the student email (`user2@uoc.edu`) by calling `GET /courses/2/enrollments` and publishes a Kafka event on `microcredential.microcredential_approved`. The Notification service logs:

```
Student user2 uoc.edu (user2@uoc.edu) has been notified by email
that microcredential 1 has been granted.
```

M7 — rejectPendingMicrocredential

Request

```
PATCH http://localhost:18085/microcredentials/2/reject
```

Response — HTTP 200

```
2
```

Microcredential id=2 transitions from `REQUESTED` → `REJECTED`. The service resolves the student email (`user3@uoc.edu`) and publishes a Kafka event on `microcredential.microcredential_rejected`. The Notification service logs:

```
Student user3 uoc.edu (user3@uoc.edu) has been notified by email
that microcredential 2 has been rejected.
```